

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 – Ağ Tabanlı Paralel Dağıtım Sistemleri
Veritabanı Performans Optimizasyonu ve İzleme

Github -> [https://github.com/Gunlsmail/BLM4522-](https://github.com/Gunlsmail/BLM4522-AgTabanlıParalelDagitimSistemleri)

[AgTabanlıParalelDagitimSistemleri](https://github.com/Gunlsmail/BLM4522-AgTabanlıParalelDagitimSistemleri)

<https://drive.google.com/drive/folders/1R1jtsEKA4ZjoR1mW4MuuvuEqVzCCSFT->

İSMAİL GÜN – 22290280
06.06.2026

İçindekiler

1. Projenin Amacı	2
2. Veritabanı Oluşturma	2
3. Tabloların Oluşturulması	3
4. Verilerin Doldurulması.....	4
5. Performans İzleme — BEFORE	5
6. Performans İzleme — AFTER.....	6
7. İndex Doğrulama	7
7.1 Yetki Testleri	7
8. Rol Tabanlı Erişim Yöntemi	9
9. Gerçek Hayattaki Kullanım Alanları	10
Sonuçlar	11

1. Projenin Amacı

Bu projede Microsoft SQL Server (MSSQL Express) kullanılarak gerçek hayat senaryosuna uygun bir e-ticaret veritabanı oluşturulmuş ve bu veritabanı üzerinde kapsamlı performans optimizasyonu teknikleri uygulanmıştır.

Projenin temel amacı; indeksiz sorguların performansını ölçmek, SQL Server Execution Plan aracını kullanarak darboğazları tespit etmek ve doğru indeks stratejileriyle sorgu performansını iyileştirmektir. Bu süreçte SQL Server'ın Dynamic Management Views (DMV) altyapısı ve Execution Plan görselleştirme araçları kullanılarak sorgu maliyetleri analiz edilmiş, tespit edilen sorunlar giderilerek sistemin verimliliği artırılmıştır.

Proje kapsamında ele alınan başlıca konular şunlardır:

- **Veritabanı İzleme:** Execution Plan ve DMV araçlarıyla sorgu performansının izlenmesi ve darboğazların tespiti
- **İndeks Yönetimi:** Sorgu hızını artırmak için doğru indekslerin oluşturulması, SQL Server önerilerinin değerlendirilmesi ve INCLUDE kolonlarıyla indeks optimizasyonu
- **Sorgu İyileştirme:** Clustered Index Scan işleminin NonClustered Index Seek işlemine dönüştürülmesiyle sorgu maliyetinin düşürülmesi
- **Veri Yöneticisi Roller:** Farklı departman kullanıcıları için rol tabanlı erişim yönetiminin uygulanması ve yetki testlerinin gerçekleştirilmesi

2. Veritabanı Oluşturma

Bu projede performans optimizasyonu testlerinin gerçekçi bir ortamda yapılabilmesi için öncelikle ECommerceDB adında yeni bir veritabanı oluşturulmuştur.

```
CREATE DATABASE ECommerceDB;  
GO  
  
USE ECommerceDB;  
GO
```

Şekil 1 : Veri Tabanı oluşturma

3. Tabloların Oluşturulması

Veritabanı içerisinde bir e-ticaret sistemini temsil eden **5 ana tablo** tanımlanmıştır. Tablolar arasında yabancı anahtar (FK) ilişkileri kurularak veri bütünlüğü sağlanmıştır. Performans testlerinin net sonuçlar üretebilmesi amacıyla tablolar **kasıtlı olarak indekssiz** bırakılmıştır; böylece indeks öncesi ve sonrası fark açıkça gözlemlenebilmiştir.

Oluşturulan tablolar ve işlevleri aşağıda verilmiştir:

- **Kategoriler:** Ürün kategorilerini tutar. KategoriID birincil anahtar olarak tanımlanmıştır.
- **Urunler:** Ürün adı, fiyat ve stok bilgilerini içerir. KategoriID ile Kategoriler tablosuna bağlıdır.
- **Musteriler:** Ad, Soyad, Email ve Sehir alanlarından oluşan müşteri kayıtlarını tutar.
- **Siparisler:** Sipariş bilgilerini içerir. MusteriID ile Musteriler tablosuna, tarih ve tutar bilgileriyle de iş mantığına bağlıdır.
- **SiparisDetaylari:** Her siparişin satır detaylarını tutar. SiparisID ve UrunID alanlarıyla çift yönlü ilişki kurulmuştur.

```
19 CREATE TABLE Kategoriler (  
20     KategoriID INT IDENTITY(1,1) PRIMARY KEY,  
21     KategoriAdi NVARCHAR(100) NOT NULL,  
22     Aciklama NVARCHAR(500)  
23 );  
24  
25  
26 CREATE TABLE Urunler (  
27     UrunID INT IDENTITY(1,1) PRIMARY KEY,  
28     UrunAdi NVARCHAR(200) NOT NULL,  
29     KategoriID INT REFERENCES Kategoriler(KategoriID),  
30     Fiyat DECIMAL(10,2) NOT NULL,  
31     Stok INT DEFAULT 0,  
32     EklenmeTarihi DATETIME DEFAULT GETDATE()  
33 );  
34  
35 CREATE TABLE Musteriler (  
36     MusteriID INT IDENTITY(1,1) PRIMARY KEY,  
37     Ad NVARCHAR(100) NOT NULL,  
38     Soyad NVARCHAR(100) NOT NULL,  
39     Email NVARCHAR(200),  
40     Sehir NVARCHAR(100),  
41     KayitTarihi DATETIME DEFAULT GETDATE()  
42 );  
43  
44 CREATE TABLE Siparisler (  
45     SiparisID INT IDENTITY(1,1) PRIMARY KEY,  
46     MusteriID INT REFERENCES Musteriler(MusteriID),  
47     SiparisTarihi DATETIME DEFAULT GETDATE(),  
48     ToplamTutar DECIMAL(10,2),  
49     Durum NVARCHAR(50) DEFAULT 'Beklemede'  
50 );  
51  
52 CREATE TABLE SiparisDetaylari (  
53     DetayID INT IDENTITY(1,1) PRIMARY KEY,  
54     SiparisID INT REFERENCES Siparisler(SiparisID),  
55     UrunID INT REFERENCES Urunler(UrunID),  
56     Miktar INT NOT NULL,  
57     BirimFiyat DECIMAL(10,2) NOT NULL  
58 );  
59 GO
```

Şekil 2 : Oluşturulan Tabloların sütunları

4. Verilerin Doldurulması

Bu projede performans testlerinin anlamlı sonuçlar üretebilmesi için veritabanına büyük miktarda gerçekçi veri yüklenmiştir. WHILE döngüsü yerine Cross Join yöntemi kullanılarak set-bazlı INSERT işlemi gerçekleştirilmiş, bu sayede veri yükleme süresi minimuma indirilmiştir.

Veri üretiminde şehir bilgileri (Ankara, İstanbul, İzmir, Bursa, Antalya, Adana, Konya, Mersin, Kayseri, Gaziantep), sipariş durumları (Beklemede, Kargoda, Teslim Edildi, İptal) ve fiyat/stok değerleri modulo işlemleriyle çeşitlendirilmiş; böylece gerçek dünya verisine yakın bir dağılım elde edilmiştir.

```
64 WITH Sayilar AS (  
65     SELECT TOP 200000  
66         ROW_NUMBER() OVER (ORDER BY a.object_id) AS n  
67     FROM sys.all_objects a  
68     CROSS JOIN sys.all_objects b  
69 )  
70 INSERT INTO Siparisler (MusteriID, SiparisTarihi, ToplanTutar, Durum)  
71 SELECT TOP 100000  
72     (n % 50000) + 1,  
73     DATEADD(DAY, -(n % 730), GETDATE()),  
74     CAST(((n * 13) % 9000 + 50) AS DECIMAL(10,2)),  
75     CASE (n % 4)  
76         WHEN 0 THEN 'Beklemede'  
77         WHEN 1 THEN 'Kargoda'  
78         WHEN 2 THEN 'Teslim Edildi'  
79         ELSE 'İptal'  
80     END  
81 FROM Sayilar;  
82 GO  
83  
84 WITH Sayilar AS (  
90 %  
Sorun bulunamadı  
Sat: 99, Kırk: 1 (427 karakter, 7 çizgi) BSL CRLF UTF-8  
Sonuçlar İletiler  
Tablo Adet  
1 Kategoriler 60  
2 Urunler 10000  
3 Musteriler 20000  
4 Siparisler 30000  
5 SiparisDetaylari 40000
```

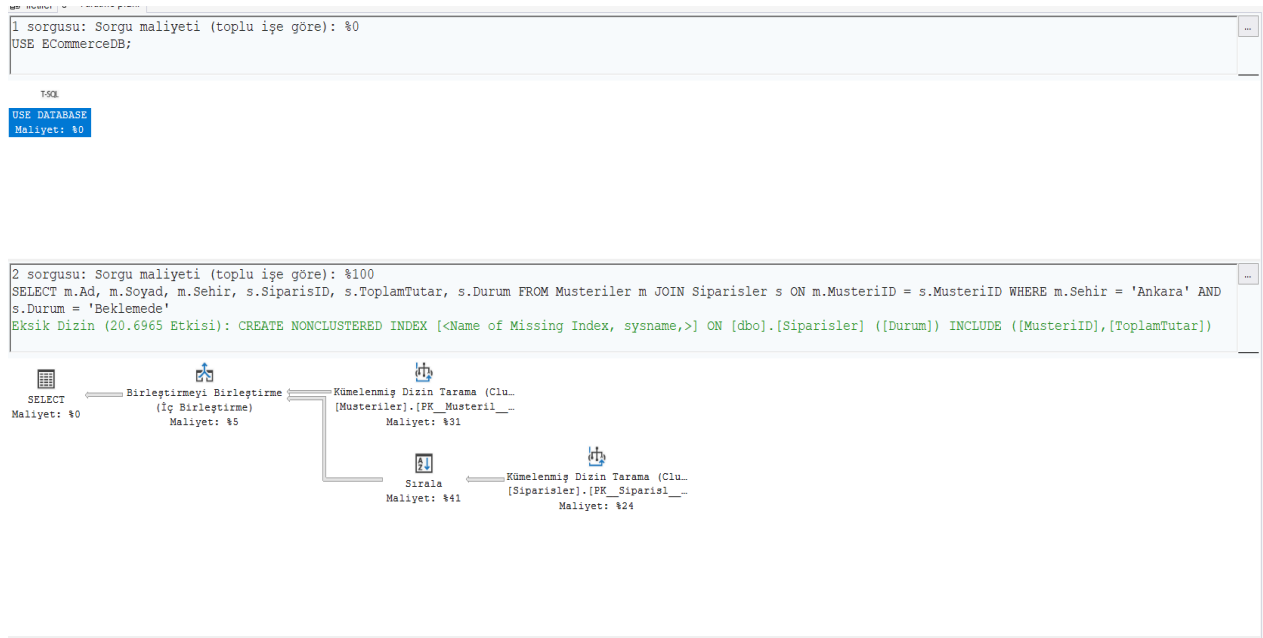
Şekil 3 : Verilerin Yapay Şekilde Doldurulması

5. Performans İzleme — BEFORE

Tablolara indeks eklenmeden önce sorgu performansı, SQL Server Management Studio'nun Tahmini Yürütme Planı (Ctrl+L) özelliği kullanılarak ölçülmüştür.

Test olarak Musteriler ve Siparisler tablolarını şehir ve sipariş durumuna göre filtreleyen bir JOIN sorgusu çalıştırılmıştır. Execution Plan incelendiğinde her iki tabloda da Kümelenmiş Dizin Tarama (Clustered Index Scan) görülmüştür. Bu durum SQL Server'ın 50.000 müşteri ve 100.000 sipariş kaydının tamamını baştan sona taradığını, yani ciddi bir performans kaybı yaşandığını göstermektedir.

Ayrıca SQL Server sarı uyarı simgesiyle eksik indeks önerisi sunmuş; Siparisler tablosundaki Durum kolonuna MusteriID ve ToplamTutar alanlarını kapsayan bir NonClustered indeks eklenmesini tavsiye etmiştir.



Şekil 4: Before aşaması performans izleme raporları

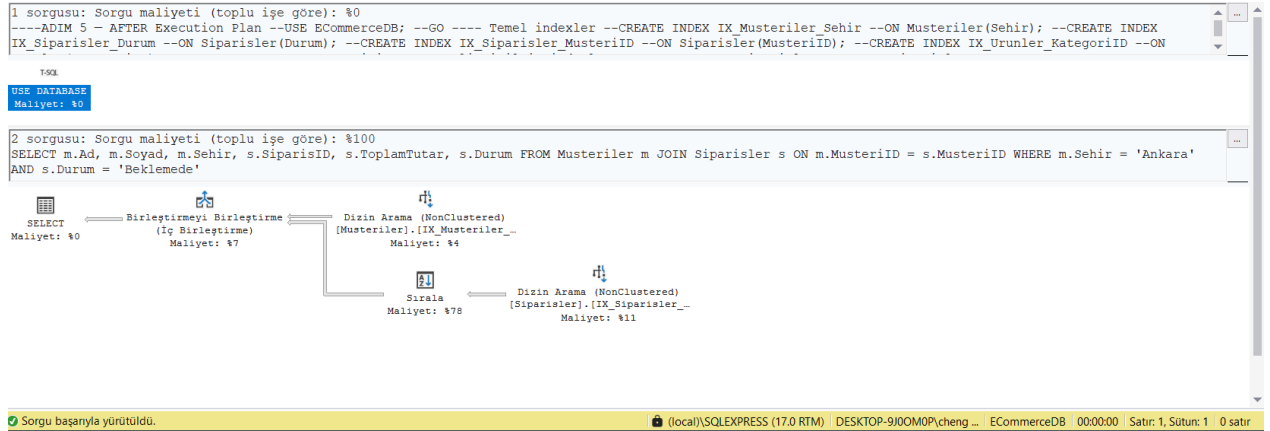
6. Performans İzleme — AFTER

Execution Plan analizinde tespit edilen performans sorunlarını gidermek amacıyla ilgili tablolara NonClustered indeksler eklenmiştir. İndeks stratejisi belirlenirken hem sorgu filtreleme koşulları hem de SQL Server'ın Execution Plan üzerinden sunduğu eksik indeks önerileri dikkate alınmıştır.

İlk aşamada dört temel indeks oluşturulmuştur. Ardından SQL Server'ın sarı uyarı simgesiyle önerdiği gelişmiş indeks yapısı incelenmiş ve mevcut indeksler INCLUDE kolonları eklenerek optimize edilmiştir. INCLUDE yapısı sayesinde indeks yalnızca filtreleme kolonunu değil, sorgu tarafından döndürülen diğer kolonları da kapsamakta; böylece tabloya ek okuma (Key Lookup) yapılması önlenmektedir.

Tablo	BEFORE	AFTER	İyileşme
Musteriler	Küm. Dizin Tarama %31	Dizin Arama %4	%87 azalma
Siparisler	Küm. Dizin Tarama %24	Dizin Arama %11	%54 azalma
Eksik indeks uyarısı	Mevcut	Giderildi	—

Şekil 5: Aşamalar arası performans karşılaştırma raporları



Şekil 6: After aşaması performans izleme raporları

7. Index Doğrulama

İndekslerin başarıyla oluşturulduğunu doğrulamak amacıyla sistem tabloları sorgulanmıştır. sys.indexes, sys.index_columns ve sys.columns view'ları birleştirilerek her tablodaki mevcut indeksler listelenmiştir.

Sorgu sonucunda 4 farklı tabloda toplam 12 indeks kaydı görüntülenmiştir. Bunların bir kısmı birincil anahtar (CLUSTERED), diğerleri ise performans amacıyla eklenen (NONCLUSTERED) indekslerdir:

	TabloAdi	IndexAdi	IndexTipi	KolonAdi
1	Musteriler	IX_Musteriler_Sehir_Final	NONCLUSTERED	Sehir
2	Musteriler	IX_Musteriler_Sehir_Final	NONCLUSTERED	Ad
3	Musteriler	IX_Musteriler_Sehir_Final	NONCLUSTERED	Soyad
4	Musteriler	PK__Musteril__7262447156D3C86C	CLUSTERED	MusterilID
5	SiparisDetaylari	PK__SiparisD__8E8164A55E113E66	CLUSTERED	DetayID
6	Siparisler	IX_Siparisler_Durum_Improved	NONCLUSTERED	Durum
7	Siparisler	IX_Siparisler_Durum_Improved	NONCLUSTERED	MusterilID
8	Siparisler	IX_Siparisler_Durum_Improved	NONCLUSTERED	ToplamTutar
9	Siparisler	IX_Siparisler_MusterilID	NONCLUSTERED	MusterilID
10	Siparisler	PK__SiparisI__C3F03BDD3E1883D3	CLUSTERED	SiparisID
11	Urunler	IX_Urunler_KategorilD	NONCLUSTERED	KategorilD
12	Urunler	PK__Urunler__623D364BB2D6850B	CLUSTERED	UrunID

Şekil 7: Index doğrulama adımları

7.1 Yetki Testleri

Rollerin doğru yapılandırıldığını kanıtlamak amacıyla EXECUTE AS USER komutu kullanılarak her kullanıcı adına çeşitli işlemler denenmiştir.

```
(5 satır etkilendi)
İleti 229, Düzey 14, Durum 5, Satır 19
The INSERT permission was denied on the object 'Musteriler', database 'ECommerceDB', schema 'dbo'.

(1 satır etkilendi)
İleti 229, Düzey 14, Durum 5, Satır 37
The DELETE permission was denied on the object 'Musteriler', database 'ECommerceDB', schema 'dbo'.

(1 satır etkilendi)

Tamamlanma süresi:2026-06-08T02:29:28.3256540+03:00
```

Şekil 11: Yetki testlerinin çıktıları

Test sonuçları incelendiğinde her rolün yalnızca tanımlanan yetkiler çerçevesinde işlem gerçekleştirebildiği görülmektedir. Yetkisiz işlem denendiğinde sistem erişimi reddetmiş ve ilgili hata mesajını döndürmüştür. Bu durum rol tabanlı erişim kontrolünün başarıyla uygulandığını kanıtlamaktadır.

	MusteriID	Ad	Soyad	Email	Sehir	KayitTarihi
1	1	Ad_1	Soyad_1	musteri1@mail.com	Istanbul	2026-06-06 14:29:07.557
2	2	Ad_2	Soyad_2	musteri2@mail.com	Izmir	2026-06-05 14:29:07.560
3	3	Ad_3	Soyad_3	musteri3@mail.com	Bursa	2026-06-04 14:29:07.560
4	4	Ad_4	Soyad_4	musteri4@mail.com	Antalya	2026-06-03 14:29:07.560
5	5	Ad_5	Soyad_5	musteri5@mail.com	Adana	2026-06-02 14:29:07.560


```

11      SELECT TOP 5 * FROM Musteriler;
12      REVERT;
13      GO
14
15      -- =====
16      -- TEST 2: READONLY - INSERT (HATA vermeli)
17      -- =====
18      EXECUTE AS USER = 'readonly_user';
19      INSERT INTO Musteriler (Ad, Soyad, Email, Sehir)
20      VALUES ('Test', 'Kullanici', 'test@mail.com', 'Ankara');
21      REVERT;
22      GO
23
24      -- =====
25      -- TEST 3: DATAENTRY - INSERT (başarılı olmalı)
26      -- =====
27      EXECUTE AS USER = 'dataentry_user';
28      INSERT INTO Musteriler (Ad, Soyad, Email, Sehir)
29      VALUES ('Dataentry', 'Kullanici', 'dataentry@mail.com', 'Istanbul');
30      REVERT;
31      GO
32
33      -- =====
34      -- TEST 4: DATAENTRY - DELETE (HATA vermeli)
35      -- =====
36      EXECUTE AS USER = 'dataentry_user';
37      DELETE FROM Musteriler WHERE Ad = 'Dataentry';
38      REVERT;
39      GO
40
41      -- =====
42      -- TEST 5: ADMIN - DELETE (başarılı olmalı)
43      -- =====
44      EXECUTE AS USER = 'admin_user';
45      DELETE FROM Musteriler WHERE Ad = 'Dataentry';
46      REVERT;
47      GO

```

Şekil 12: Yetki testlerinin Kodları ve İncelenmesi

```

-- =====
-- 3) KULLANICILARI ROLLERE ATAATAMA
-- =====
ALTER ROLE rol_readonly ADD MEMBER readonly_user;
ALTER ROLE rol_dataentry ADD MEMBER dataentry_user;
ALTER ROLE rol_admin ADD MEMBER admin_user;
GO

-- =====
-- 4) KONTROL - Roller ve kullanıcılar
-- =====
SELECT
    r.name AS RolAdi,
    m.name AS KullaniciAdi
FROM sys.database_role_members rm
JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id
JOIN sys.database_principals m ON rm.member_principal_id = m.principal_id
WHERE r.name LIKE 'rol_%'
ORDER BY r.name;
GO

```

Şekil 9: Kullanıcılara rol atama aşamaları

8. Rol Tabanlı Erişim Yöntemi

Veritabanı güvenliğinin sağlanması amacıyla farklı departman kullanıcılarını temsil eden 3 farklı rol tanımlanmış ve bu rollere uygun yetkiler atanmıştır. Rol tabanlı erişim kontrolü (RBAC) yaklaşımı benimsenerek her kullanıcının yalnızca ihtiyaç duyduğu işlemleri gerçekleştirebilmesi hedeflenmiştir.

Adım adım kullanıcı oluşturma ve atamalar aşağıda görünmektedir:

```
-- =====
-- 1) KULLANICILARI OLUŞTUR
-- =====

-- Önce login oluştur (SQL Server seviyesi)
CREATE LOGIN readonly_login
    WITH PASSWORD = 'Readonly123!';

CREATE LOGIN dataentry_login
    WITH PASSWORD = 'Dataentry123!';

CREATE LOGIN admin_login
    WITH PASSWORD = 'Admin123!';
GO

-- Sonra DB kullanıcısı oluştur (ECommerceDB seviyesi)
CREATE USER readonly_user FOR LOGIN readonly_login;
CREATE USER dataentry_user FOR LOGIN dataentry_login;
CREATE USER admin_user FOR LOGIN admin_login;
GO

-- =====
-- 2) ROL OLUŞTUR VE YETKİ VER
-- =====

-- READONLY ROLÜ
CREATE ROLE rol_readonly;
GRANT SELECT ON Musteriler TO rol_readonly;
GRANT SELECT ON Siparisler TO rol_readonly;
GRANT SELECT ON SiparisDetaylari TO rol_readonly;
GRANT SELECT ON Urunler TO rol_readonly;
GRANT SELECT ON Kategoriler TO rol_readonly;

-- DATAENTRY ROLÜ
CREATE ROLE rol_dataentry;
GRANT SELECT, INSERT, UPDATE ON Musteriler TO rol_dataentry;
GRANT SELECT, INSERT, UPDATE ON Siparisler TO rol_dataentry;
GRANT SELECT, INSERT, UPDATE ON SiparisDetaylari TO rol_dataentry;
GRANT SELECT ON Urunler TO rol_dataentry;
GRANT SELECT ON Kategoriler TO rol_dataentry;

-- ADMIN ROLÜ
CREATE ROLE rol_admin;
GRANT CONTROL ON SCHEMA::dbo TO rol_admin;
GO
```

Şekil 8: Kullanıcı oluşturma adımları

9. Gerçek Hayattaki Kullanım Alanları

Veritabanı performans optimizasyonu günümüzde her ölçekteki sistemde kritik bir rol oynamaktadır:

E-Ticaret Sistemleri: Amazon, Trendyol gibi platformlarda milyonlarca ürün ve sipariş kaydı anlık olarak sorgulanmaktadır. İndeks optimizasyonu olmadan bu sorguların saniyeler içinde tamamlanması mümkün değildir.

Bankacılık ve Finans: Müşteri hesap hareketleri, kredi başvuruları ve risk analizleri yoğun sorgu trafiği oluşturmaktadır. Doğru indeks stratejileri bu sistemlerde milisaniye düzeyinde yanıt süresi sağlamaktadır.

Sağlık Sistemleri: Hasta kayıtları, tahlil sonuçları ve ilaç takibi gibi kritik verilerin hızlı ve güvenli sorgulanması hasta güvenliği açısından hayati önem taşımaktadır.

Rol Tabanlı Erişim: Bankada müşteri temsilcisi hesap bakiyesini görebilir ancak silemez. Doktor hasta kayıtlarına erişebilir ancak fatura bilgilerini düzenleyemez. Bu projede uygulanan RBAC modeli tam olarak bu gerçek hayat senaryolarını yansıtmaktadır.

Sonuçlar

Bu projede Microsoft SQL Server ortamında gerçek bir e-ticaret senaryosu üzerinde kapsamlı bir veritabanı performans optimizasyonu çalışması gerçekleştirilmiştir. Proje başlangıcında yaklaşık 360.000 satır veri içeren ECommerceDB yapay olarak oluşturulan veritabanı ve indekssiz yapıda sorgu performansı ölçülmüştür.

Execution Plan analizleri sonucunda indeks eklenmeden önce her iki tabloda da **Kümelenmiş Dizin Tarama (%100 maliyet)** tespit edilmiştir. Uygulanan NonClustered indeks stratejisi ile bu değer Musteriler tablosunda **%4'e**, Siparisler tablosunda ise **%11'e** düşürülmüştür. SQL Server'ın kendi önerdiği eksik indeks uyarısı da bu süreçte tamamen giderilmiştir.

Rol tabanlı erişim yönetimi kapsamında 3 farklı kullanıcı rolü tanımlanmış ve yetki testleriyle her rolün yalnızca tanımlı işlemleri gerçekleştirebildiği kanıtlanmıştır.